Name : Baman Jaynilkumar
Section : A - 2
Registration no. : 20253074
Serial no. : 41

Assignment - 6

1.Your task is to write a C program that accepts two polynomials as input, performs the addition, and outputs the resulting polynomial.

Code :

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node* next;
};

void insert(struct Node** poly, int coeff, int exp) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->coeff = coeff;
    temp->exp = exp;
    temp->next = NULL;

    if (*poly == NULL) {
```

```
        *poly = temp;
        return;
    }
    struct Node* current = *poly;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = temp;
}

void display(struct Node* poly) {
    if (poly == NULL) {
        printf("0\n");
        return;
    }
    while (poly != NULL) {
        printf("%dx^%d", poly->coeff, poly->exp);
        poly = poly->next;
        if (poly != NULL) printf(" + ");
    }
    printf("\n");
}

struct Node* addPolynomials(struct Node* p1, struct Node* p2) {
    struct Node* result = NULL;
    while (p1 != NULL && p2 != NULL) {
        if (p1->exp > p2->exp) {
            insert(&result, p1->coeff, p1->exp);
            p1 = p1->next;
        } else if (p1->exp < p2->exp) {
            insert(&result, p2->coeff, p2->exp);
```

```
            p2 = p2->next;
        } else {
            insert(&result, p1->coeff + p2->coeff, p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
    }
    while (p1 != NULL) {
        insert(&result, p1->coeff, p1->exp);
        p1 = p1->next;
    }
    while (p2 != NULL) {
        insert(&result, p2->coeff, p2->exp);
        p2 = p2->next;
    }
    return result;
}

int main() {
    struct Node* poly1 = NULL;
    struct Node* poly2 = NULL;

    // Example: 3x^2 + 5x^1 + 6
    insert(&poly1, 3, 2);
    insert(&poly1, 5, 1);
    insert(&poly1, 6, 0);

    // Example: 6x^1 + 8
    insert(&poly2, 6, 1);
    insert(&poly2, 8, 0);
```

```
    printf("Polynomial 1: "); display(poly1);
    printf("Polynomial 2: "); display(poly2);

    struct Node* sum = addPolynomials(poly1, poly2);
    printf("Sum: "); display(sum);

    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 1 1.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./1
Polynomial 1: 3x^2 + 5x^1 + 6x^0
Polynomial 2: 6x^1 + 8x^0
Sum: 3x^2 + 11x^1 + 14x^0
```

2.Suppose you are working in a software company that specializes in mathematical computations. Your current task is to develop a program that can perform polynomial addition. Polynomials are algebraic expressions consisting of variables and coefficients. You need to create a program where the user can input two polynomials, and the program will return their multiplication.

Code :

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node* next;
};

void insert(struct Node** poly, int coeff, int exp) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->coeff = coeff;
```

```
    temp->exp = exp;
    temp->next = NULL;

    if (*poly == NULL || (*poly)->exp < exp) {
        temp->next = *poly;
        *poly = temp;
        return;
    }

    struct Node* current = *poly;
    while (current->next != NULL && current->next->exp >= exp) {
        current = current->next;
    }

    if (current->exp == exp) {
        current->coeff += coeff;
        free(temp);
    } else {
        temp->next = current->next;
        current->next = temp;
    }
}

void display(struct Node* poly) {
    if (poly == NULL) {
        printf("0\n");
        return;
    }
    while (poly != NULL) {
        printf("%dx^%d", poly->coeff, poly->exp);
        poly = poly->next;
```

```
        if (poly != NULL) printf(" + ");
    }
    printf("\n");
}

struct Node* multiplyPolynomials(struct Node* p1, struct Node* p2)
{
    struct Node* result = NULL;
    struct Node* ptr1 = p1;
    struct Node* ptr2 = p2;

    while (ptr1 != NULL) {
        ptr2 = p2;
        while (ptr2 != NULL) {
            int coeff = ptr1->coeff * ptr2->coeff;
            int exp = ptr1->exp + ptr2->exp;
            insert(&result, coeff, exp);
            ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
    }
    return result;
}

int main() {
    struct Node* poly1 = NULL;
    struct Node* poly2 = NULL;

    // Example: 2x^2 + 3x^1
    insert(&poly1, 2, 2);
    insert(&poly1, 3, 1);
```

```
    // Example: 4x^1 + 5
    insert(&poly2, 4, 1);
    insert(&poly2, 5, 0);

    printf("Polynomial 1: "); display(poly1);
    printf("Polynomial 2: "); display(poly2);

    struct Node* product = multiplyPolynomials(poly1, poly2);
    printf("Product: "); display(product);

    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> notepad 2.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 2 2.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./2
Polynomial 1: 2x^2 + 3x^1
Polynomial 2: 4x^1 + 5x^0
Product: 8x^3 + 22x^2 + 15x^1
```

3.WAP to implement Stack using an array:

a.Push an element onto a stack.

b.Pop the stack

c.Display the contents of the stack.

Code :

```
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1) {
        printf("Stack Overflow!\n");
```

```
    } else {
        top++;
        stack[top] = value;
        printf("%d pushed to stack.\n", value);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow!\n");
    } else {
        printf("%d popped from stack.\n", stack[top]);
        top--;
    }
}

void display() {
    if (top == -1) {
        printf("Stack is empty.\n");
    } else {
        printf("Stack contents: ");
        for (int i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main() {
    push(10);
    push(20);
```

```
    push(30);
    display();
    pop();
    display();
    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> notepad 3.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 3 3.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./3
10 pushed to stack.
20 pushed to stack.
30 pushed to stack.
Stack contents: 30 20 10
30 popped from stack.
Stack contents: 20 10
```

4.WAP to implement Stack using linked list:

a. Push a element onto a stack
b. Pop the stack
c. Display the contents of the stack.

Code :

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push(int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed to stack.\n", value);
}

void pop() {
    if (top == NULL) {
        printf("Stack Underflow!\n");
    } else {
        struct Node* temp = top;
        printf("%d popped from stack.\n", top->data);
        top = top->next;
        free(temp);
    }
}

void display() {
    if (top == NULL) {
        printf("Stack is empty.\n");
    } else {
        struct Node* temp = top;
        printf("Stack contents: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
```

```
}

int main() {
    push(100);
    push(200);
    display();
    pop();
    display();
    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> notepad 4.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 4 4.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./4
100 pushed to stack.
200 pushed to stack.
Stack contents: 200 100
200 popped from stack.
Stack contents: 100
```

5.WAP to implement Queue using an array:

a. **Insert an element into the queue**
b. **Delete an element from the queue**
c. **Display the contents of the queue**

Code :

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

void insert(int value) {
    if (rear == MAX - 1) {
```

```
        printf("Queue is full!\n");
    } else {
        if (front == -1) front = 0;
        rear++;
        queue[rear] = value;
        printf("%d inserted into queue.\n", value);
    }
}

void delete() {
    if (front == -1 || front > rear) {
        printf("Queue is empty!\n");
    } else {
        printf("%d deleted from queue.\n", queue[front]);
        front++;
    }
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
    } else {
        printf("Queue contents: ");
        for (int i = front; i <= rear; i++) {
            printf("%d ", queue[i]);
        }
        printf("\n");
    }
}

int main() {
```

```
    insert(1);
    insert(2);
    insert(3);
    display();
    delete();
    display();
    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> notepad 5.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 5 5.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./5
1 inserted into queue.
2 inserted into queue.
3 inserted into queue.
Queue contents: 1 2 3
1 deleted from queue.
Queue contents: 2 3
```

6. WAP to implement Queue using linked list:

a. Insert an element into the queue
b. Delete an element from the queue
c. Display the contents of the queue

Code :

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;
```

```
void insert(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL) {
        front = rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted into queue.\n", value);
}

void delete() {
    if (front == NULL) {
        printf("Queue is empty!\n");
    } else {
        struct Node* temp = front;
        printf("%d deleted from queue.\n", front->data);
        front = front->next;
        if (front == NULL) rear = NULL;
        free(temp);
    }
}

void display() {
    if (front == NULL) {
        printf("Queue is empty.\n");
    } else {
```

```
        struct Node* temp = front;
        printf("Queue contents: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
}

int main() {
    insert(50);
    insert(60);
    display();
    delete();
    display();
    return 0;
}
```

```
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> notepad 6.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> gcc -o 6 6.c
PS C:\Users\Baman Jaynilkumar\Desktop\DSA assignment proofs\w6a1> ./6
50 inserted into queue.
60 inserted into queue.
Queue contents: 50 60
50 deleted from queue.
Queue contents: 60
```